

0.80. x et y rangés ensemble : struct

`struct Position { uint8_t x, y; }` : la colonne et la ligne d'une lettre sous un seul nom, `joueur.x` et `joueur.y`.

1. Ce qu'il faut comprendre

Deux variables qui vont toujours ensemble — la colonne et la ligne d'une lettre — peuvent être rangées **sous un seul nom**.

Ce qui est nouveau ici : `struct`. `struct Position { uint8_t x, y; };` crée un **nouveau type**, `Position`, qui contient deux octets : `x` et `y`. Puis `Position joueur;` crée une variable de ce type.

Le point `.` atteint une partie : `joueur.x` est la colonne du joueur, `joueur.y` sa ligne. On s'en sert comme de n'importe quelle variable : `joueur.x = 10;`, `deplace_croix(joueur.x, joueur.y, ...)`.

Pourquoi c'est utile : avec plusieurs lettres, `Position joueur;` et `Position ennemi;` se lisent mieux que `x`, `y`, `xb`, `yb`. On ne mélange plus les variables d'une lettre avec celles d'une autre.

La console range toujours la position dans `joueur.x` et `joueur.y` : ce qu'elle sait faire avec une variable (le 0.23), elle le sait avec une partie de `struct`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1775 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Le nouveau type, morceau par morceau :  
//  
// struct Position { uint8_t x, y; };  
// |           |           |  
// |           |           +-- ses parties : deux octets, x et y  
// |           +----- son nom : Position  
// +----- « struct » : on crée un type à nous
```

2. UN ENREGISTREMENT — POSITION

```
struct Position { uint8_t x, y; };
```

3. UNE VARIABLE — JOUEUR

```
Position joueur;      // une variable de type Position : joueur.x et joueur.y
```

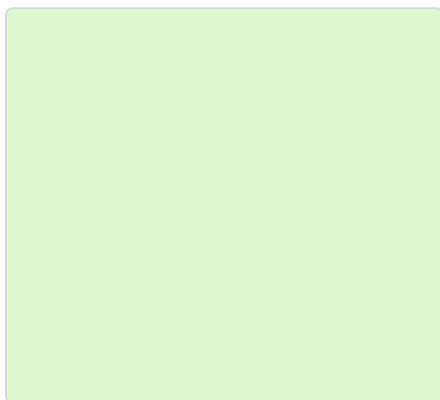
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    joueur.x = 10;      // le point atteint une partie : la colonne...
    joueur.y = 0;      // ...et la ligne. Départ : (10, 0).

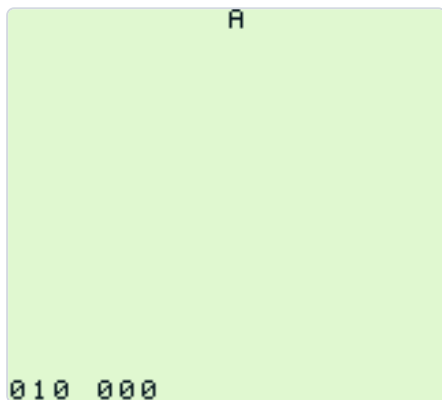
    while (true) {
        image();
        deplace_croix(joueur.x, joueur.y, ALPHABET[0], 250); // la console range dans joueur.x, joueur.y
        nombre(0, 17, joueur.x);
        nombre(4, 17, joueur.y);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A en haut de l'écran, qui suit la croix ; sa position, joueur.x et joueur.y, en bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A part de (10, 0)
- ✓ la croix le déplace : joueur.x diminue, joueur.y augmente (6, 2)
- ✓ il est à la place que disent joueur.x et joueur.y

Ensuite : **0.80.1. x et y rangés ensemble : struct — la boucle dans un fichier** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++